EXVUL

SMART CONTRACT AUDIT REPORT

Bibo Smart Contracts

SEPTEMBER 2026

Contents

1. EXECUTIVE SUMMARY **3**
1.1 Methodology . . . 3
1.2 Scope and Limitations . . . 5

2. FINDINGS OVERVIEW **6**
2.1 Project Information . . . 6
2.2 Summary . . . 6
2.3 Key Findings . . . 7
2.4 Validation Summary . . . 7

3. DETAILED DESCRIPTION OF FINDINGS **8**
3.1 The launch economics commitment omits anti-snipe terms . . . 8

4. CONCLUSION **12**

5. APPENDIX **13**
5.1 Validation Evidence . . . 13
5.2 Security Assessment Summary . . . 13
5.3 Fork Provenance . . . 14

6. DISCLAIMER **15**

7. REFERENCES **16**

8. ABOUT EXVUL SECURITY **17**

1. EXECUTIVE SUMMARY

ExVul Web3 Security reviewed the **Bibo** smart contract implementation in the project's published audit repository. Bibo's official website is `bibo.trade`. The assessment covered the launch factory, bonding curve, launch token, fee escrow, buyback vault, graduation path, Uniswap V4 hook integration, launch-and-buy periphery, and protocol-fee components.

No Critical, High, or Medium vulnerability was identified at the audited commit. One Low-severity correctness issue was reported and is marked **Fixed**. No reviewed path allowed an ordinary user to obtain positive profit, cause victim asset loss, create unbacked accounting, bypass a privileged role, or persist attacker-benefiting state corruption.

The reviewed launchpad is substantially a naming fork of Pons V2. Thirteen launchpad files are source-equivalent after consistent identifier normalization to the chain-verified Pons V2 bundle, and the scoped protocol-fee files match the referenced Uniswap protocol-fees baseline. Upstream similarity informed review prioritization but was not treated as evidence of safety; Bibo's integration paths were independently traced and tested.

The final validation suite passed 212 of 212 tests across 26 suites. The continuous integration profile executed 10,000 cases per fuzz test and 256 invariant runs at depth 128. The Bibo-only coverage profile reported 96.44% line coverage, 100.00% function coverage, and 67.64% branch coverage.

1.1 Methodology

The assessment combined bottom-up manual review with executable validation. The process included:

- complete function and entry-point inventory, including view and pure functions;
- state-transition, authorization, trust-boundary, and external-call analysis;
- reverse-operation and accounting-invariant review across buy, sell, fee, rescue, and graduation paths;
- fork-delta comparison against Pons V2 and Uniswap protocol-fee sources;
- adversarial unit, fuzz, invariant, and integration testing under the repository compiler matrix; and
- strict exploitability validation requiring ordinary-user reachability and concrete asset, state, or authorization impact.

Risk ratings follow the OWASP likelihood and impact model. Likelihood estimates whether a weakness can be reached and exploited under realistic conditions. Impact measures the resulting technical and business loss. Severity combines these factors into Critical, High, Medium, Low, or Informational classifications.

Likelihood	Informational	Low	Medium	High
High	INFO	MEDIUM	HIGH	CRITICAL
Medium	INFO	LOW	MEDIUM	HIGH
Low	INFO	LOW	LOW	MEDIUM

IMPACT

Table 1.1 Overall Risk Severity

1.2 Scope and Limitations

The primary review scope consisted of 22 Solidity files and 6,310 lines identified by the repository's audit scope. The function inventory contained 293 functions, including 213 externally reachable or interface entry points.

The repository does not contain a production deployment manifest or deployed contract addresses. This source review therefore does not establish deployed-bytecode equivalence and does not verify the live holders of owner, fee-setter, releaser, sweep-operator, PoolManager, PositionManager, or Permit2 privileges. Those deployment controls should be verified before production acceptance.

Review Area	Assessment Focus
Launch and Curve Logic	Launch configuration, deterministic deployment, pricing, buy and sell accounting, anti-snipe taxation, slippage, and graduation triggers.
Asset and Fee Custody	Native and ERC-20 custody, fee accrual, escrow claims, rescue paths, buyback conversion, and accounting backing.
Uniswap V4 Integration	Hook deltas, settlement, pool initialization, liquidity minting, protocol-fee policy, and collection routing.
Authorization	Owner and operator boundaries, whitelisting, immutable dependencies, forwarder trust, and role transitions.
Adversarial Validation	Reentrancy, extreme arithmetic, self-recipient trades, partial fills, dust behavior, fuzzing, and invariants.

Table 1.2 Assessment Coverage

2. FINDINGS OVERVIEW

2.1 Project Information

Project Name	Audit Time	Language
Bibo	15/09/2026 - 15/09/2026	Solidity

Official Website	https://bibo.trade
Repository	https://github.com/bibodottrade/bibo-contracts-audit
Commit Hash	`c509a36bf42c71c912b479b3043af1f6b3560098`

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	1
INFO	0



2.3 Key Findings

Severity	Findings Title	Status
LOW	The launch economics commitment omits anti-snipe terms	Fixed

Table 2.3 Key Audit Findings

2.4 Validation Summary

The review added five adversarial regression tests covering the reported commitment defect, fee-rescue callback behavior, curve self-recipient trades, and extreme accepted launch configurations. All five tests passed. Existing protocol-fee dust regressions also passed.

The final deterministic CI run passed 212 of 212 tests across 26 suites. No candidate path met the verified exploit gate for normal-user profit, victim-state damage, authorization bypass, or persistent attacker-benefiting corruption.

3. DETAILED DESCRIPTION OF FINDINGS

3.1 The launch economics commitment omits anti-snipe terms

SEVERITY: LOW **STATUS:** Fixed

PATH:

`src/bibo/src/v2/BiboLaunchFactory.sol`

`src/bibo/src/v2/BiboBondingCurve.sol`

DESCRIPTION:

The creator-facing `expectedEconomics` commitment is intended to prevent owner-controlled launch terms from changing between preview and execution. However, the digest at the audited commit does not include `snipeTaxStartBps` or `snipeTaxSeconds`, even though both values are owner-mutable and snapshotted by every new curve during initialization.

```
// BiboLaunchFactory.sol:666-686
function _economicsDigest(
    LaunchConfig memory config,
    FeePolicySnapshot memory policy,
    uint256 phantomQuote,
    uint256 graduationThreshold
) private pure returns (bytes32) {
    return keccak256(
        abi.encode(
            phantomQuote,
            graduationThreshold,
            config.supply,
            config.curveFeeBps,
            config.poolFee,
            config.tickSpacing,
            policy.protocolFeeShareBps,
            policy.buybackBurnBps,
            policy.hookFeeBps,
            policy.maxInternalPriceImpactBps
        )
    );
}
```

An owner can therefore change only the two anti-snipe values after a creator obtains a digest. The creator's old digest remains valid, while the launched curve stores and applies the new schedule:

```
// BiboBondingCurve.sol:269-284
function initialize(address token_) external onlyFactory {
    ...
    launchTimestamp = block.timestamp;
    snipeTaxStartBps = IBiboSnipeTax(factory).snipeTaxStartBps();
    snipeTaxSeconds = IBiboSnipeTax(factory).snipeTaxSeconds();
}
```

IMPACT:

This is a commitment-integrity defect. A creator can approve one preview while the resulting launch uses different anti-snipe terms. The security impact is bounded because changing either omitted value requires the factory owner, the creator and creator-fee recipient are automatically exempt, the atomic launch-and-buy recipient is also exempt, and any undeclared buyer can enforce `minTokensOut` so that an unexpectedly taxed purchase reverts atomically.

The compiler-exact regression confirmed that changing only these two values left the digest unchanged and caused a newly launched curve to snapshot the modified values. It did not produce normal-user extraction, victim loss, unbacked accounting, or an authorization bypass.

RECOMMENDATIONS:

Version the economics commitment and include both anti-snipe parameters in the committed preimage. Both the preview and launch paths must use the same field order and values. A representative remediation pattern is shown below:

```
function _economicsDigest(
    LaunchConfig memory config,
    FeePolicySnapshot memory policy,
    uint256 phantomQuote,
    uint256 graduationThreshold,
+   uint256 snipeTaxStartBps_,
+   uint256 snipeTaxSeconds_
) private pure returns (bytes32) {
    return keccak256(
        abi.encode(
+           uint256(2), // commitment version
            phantomQuote,
            graduationThreshold,
            config.supply,
            config.curveFeeBps,
            config.poolFee,
            config.tickSpacing,
            policy.protocolFeeShareBps,
            policy.buybackBurnBps,
            policy.hookFeeBps,
-           policy.maxInternalPriceImpactBps
+           policy.maxInternalPriceImpactBps,
+           snipeTaxStartBps_,
+           snipeTaxSeconds_
        )
    );
}
```

Add regressions asserting that changing either anti-snipe parameter invalidates every old nonzero commitment and that an unchanged configuration produces an identical digest in the preview and launch paths.

RESOLUTION:

The issue status is **Fixed**. The remediation commits both anti-snipe terms to the creator-approved launch economics and preserves a shared digest construction across preview and execution.

4. CONCLUSION

ExVul Web3 Security reviewed the Bibo smart contract system and identified one Low-severity correctness issue. No Critical, High, or Medium vulnerability was identified. The reported Low-severity issue is marked **Fixed**, and the audit result is therefore considered **PASSED** for the reviewed source scope.

The assessment did not identify an ordinary-user path to positive profit, victim asset loss, persistent attacker-benefiting state corruption, unbacked accounting, or privileged-role bypass. Deployment acceptance should still include bytecode verification and confirmation of all live administrative and integration addresses.

5. APPENDIX

5.1 Validation Evidence

Validation	Result
Final CI	212 of 212 tests passed across 26 suites.
Fuzz Testing	10,000 cases per fuzz test in the deterministic CI profile.
Invariant Testing	256 runs at depth 128.
Adversarial Regressions	5 of 5 new tests passed.
Bibo Coverage	96.44% lines, 100.00% functions, and 67.64% branches.
Compiler	Repository compiler matrix, including solc 0.8.35 for the Bibo V2 scope.

5.2 Security Assessment Summary

Category	Assessment	Result
Access Control	Factory, hook, escrow, vault, forwarder, and protocol-fee role boundaries	No exploitable issue identified
Reentrancy	Native and ERC-20 callback paths, rescue operations, trading, and fee payouts	No exploitable issue identified
Asset Accounting	Tracked reserves, physical balances, fees, donations, partial fills, and refunds	No exploitable issue identified
Arithmetic	Pricing, rounding, terminal fills, graduation math, and extreme configurations	No exploitable issue identified
External Integration	Uniswap V4 hooks, PoolManager settlement, PositionManager minting, and protocol fees	No exploitable issue identified
Commitment Integrity	Creator-approved launch economics and mutable protocol parameters	One Low issue, Fixed
Deployment Consistency	Production bytecode and live role holders	Requires deployment verification

5.3 Fork Provenance

The effective fork comparison used two Pons references: an older public source snapshot and a later chain-verified Pons V2 bundle. The apparent differences from the older snapshot, including deterministic deployment, anti-snipe accounting, supply and time snapshots, and fee-escrow hardening, are already present in the later verified Pons V2 lineage.

After consistent Bibo-to-Pons identifier normalization, thirteen launchpad files were source-equivalent to the later verified Pons bundle. `BiboFeeEscrow.sol` was semantically equivalent except for a comment-only name reference. `BiboLaunchAndBuy.sol` matched historical Pons source after normalization. All seven scoped files under `src/protocol-fees/src` matched the reviewed Uniswap protocol-fees baseline.

6. DISCLAIMER

This report is subject to the terms and conditions, including the description of services, confidentiality, disclaimer, and limitation of liability, set forth in the applicable Services Agreement or scope of services. The report may be used only to the extent permitted by those terms and may not be transmitted, disclosed, referred to, or relied upon by another person without ExVul's prior written consent.

This report is not an endorsement or disapproval of any project, team, product, token, asset, business model, or legal position. It does not provide a warranty or guarantee that the reviewed technology is free of defects. The assessment is limited to the identified source scope and review period, and later code, deployment configuration, dependencies, governance decisions, or operational changes may alter the security posture.

This report must not be used as investment advice. Blockchain systems and cryptographic assets involve ongoing technical and operational risk. Project teams and users remain responsible for their own due diligence, deployment verification, monitoring, incident response, and continuous security practices.

7. REFERENCES

[1] OWASP. Risk Rating Methodology.
https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[2] MITRE. CWE-682: Incorrect Calculation.
https://cwe.mitre.org/data/definitions/682.html

[3] Solidity Documentation. Security Considerations.
https://docs.soliditylang.org/en/latest/security-considerations.html

[4] Bibo. Official Website.
https://bibo.trade

[5] Bibo. Audited Source Repository.
https://github.com/bibodottrade/bibo-contracts-audit

8. ABOUT EXVUL SECURITY

ExVul is a Web3 security firm focused on securing decentralized protocols and the infrastructure supporting them. Its services include smart contract audits, blockchain protocol reviews, wallet assessments, and proactive Web3 penetration testing.

The team combines experience from global technology and blockchain security organizations. ExVul has supported projects across EVM, Solana, Aptos, Sui, and other ecosystems, with work spanning application logic, asset accounting, consensus and client software, integrations, and operational security.

EXVUL

Contact

Website
www.exvul.com

Email
contact@exvul.com

Twitter
@EXVULSEC

Github
github.com/EXVUL-Sec

ExVul